Software Process Improvement

Individual Assignment

Personal Software Process (PSP)

Part 2: Individual Work

(i) Record of Previous Work • (ii) Student Grade Management System (Python 3)

Name:	Zelalem Fisseha
ID No.:	BITS UGR/0177
Section:	_____
Submitted to:	_____
Date of submission:	_____

Source code (GitHub repository)
https://github.com/Zelalem-Fisseha/Zelalem_Fisseha_Bits_UGR_0177
Branch **main**, commit `bed6485`. Click the link to open the repository.

Contents

Part A. PSP Report . . . 3

1. Introduction . . . 3

2. Part 2 (i): Record of Previous Work . . . 3

3. Part 2 (ii): Student Grade Management System with PSP . . . 4

4. Reflection: did PSP make my workflow better or worse? . . . 9

Part B. User Guide . . . 10

1. What the program does . . . 10

2. What you need . . . 10

3. How to run it . . . 10

4. How to use it . . . 10

5. Input rules . . . 10

6. Grading scale . . . 10

7. Sample run . . . 11

8. Running the tests . . . 11

Part C. PSP Logs (PSP_Logs.xlsx) . . . 12

C.1 Sheet "Job Number Log" . . . 12

C.2 Sheet "Daily Time Log" . . . 13

C.3 Sheet "Part 2 Summary" . . . 14

Part D. Source Code . . . 15

Repository contents . . . 15

How to get and run the code . . . 15

D.1 source/grade_manager.py . . . 16

D.2 source/test_grade_manager.py . . . 18

This file puts the whole submission in one place. Part A is the PSP report, Part B is the user guide of the program, Part C is the Excel workbook *PSP_Logs.xlsx* (the logs, with the values its formulas calculate) and Part D is the source code, taken from the GitHub repository named on the cover. The original files (*PSP_Report.docx*, *User_Guide.docx*, *PSP_Logs.xlsx*) are also in that repository.

Part A. PSP Report

Software Process Improvement - Individual Assignment, Part 2

1. Introduction

This report is my individual work for the Personal Software Process (PSP) assignment. In Part (i) I recorded a project I had already finished, the Simple ATM Banking System in C++, using my Daily Time Log. In Part (ii) I built a small Student Grade Management System in Python and followed PSP while doing it: I split the work into jobs, estimated each job before starting it, logged my time every day and updated the Job Number Log after each job.

All times are in minutes. Delta Time is the time between start and stop minus interruptions. The same five processes are used in both parts so the To Date figures carry over from Part (i) to Part (ii): Requirements (counted in requirements), Design (functions), Code (LOC, counted as non-blank lines that are not comments), Test (test cases) and Documentation (pages).

The Excel file PSP_Logs.xlsx has the same logs with formulas (Delta Time, rates and To Date values), so the numbers in this report can be checked there. The workbook is reproduced in Part C.

2. Part 2 (i): Record of Previous Work

2.1 The project

Project: Simple ATM Banking System (C++, console application)

A console ATM simulator written in C++ as a small individual project earlier in the programme. The user logs in with a 4-digit PIN (3 tries), then uses a menu to check the balance, deposit, withdraw, see the last 5 transactions and change the PIN. Data was kept in memory only (no files).

I divided the work I did on this project into the same five kinds of job I use in Part (ii). Each job got a job number (101-105). The Daily Time Log in section 2.2 is what I recorded while doing the project, and the Job Number Log in section 2.3 is filled from it.

2.2 Daily Time Log - previous project

Date	Job #	Start	Stop	Interruption (min)	Delta Time (min)	Activity	Comments	C	U
05/10/2026	101	19:10	19:58	3	45	Requirements	Read the assignment sheet, listed the ATM features (PIN, balance, deposit, withdraw...)	X	8
05/10/2026	102	20:05	20:50	0	45	Design	Flowchart for main menu and PIN check		
06/10/2026	102	18:30	19:30	10	50	Design	Function list + pseudocode for deposit/withdraw. Phone call from home	X	7
06/10/2026	103	19:40	21:15	5	90	Code	Account struct, menu loop, PIN check		
07/10/2026	103	14:00	15:50	15	95	Code	deposit / withdraw / balance functions. Friend came to borrow notes		
07/10/2026	103	20:10	21:30	0	80	Code	Transaction history (array of last 5), fixed compile errors		
08/10/2026	103	09:00	10:20	15	65	Code	Input checks, change PIN, cleaned up code. Break for tea	X	214
08/10/2026	104	10:30	11:45	5	70	Test	Ran 9 test cases from my test table. Fixed: withdraw accepted a negative amount	X	9
09/10/2026	105	16:00	17:05	0	65	Documentation	Report: introduction, features, screenshots		
09/10/2026	105	20:00	20:50	5	45	Documentation	User manual part + formatting	X	3
			Total	**58**	**650**				

Table 1. Daily Time Log for the Simple ATM Banking System (C = job completed, U = units completed).

2.3 Job Number Log - previous project

Job #	Date	Process	Estimated		Actual			To Date					Description (unit)
			Time	Units	Time	Units	Rate	Time	Units	Rate	Max	Min	
101	05/10/2026	Requirements			45	8	5.63	45	8	5.63	5.63	5.63	ATM project - read assignment and list requirements (requirements)
102	05/10/2026	Design			95	7	13.57	95	7	13.57	13.57	13.57	ATM project - flowchart and function design (functions)
103	06/10/2026	Code	240		330	214	1.54	330	214	1.54	1.54	1.54	ATM project - coding in C++ (atm.cpp) (LOC)
104	08/10/2026	Test			70	9	7.78	70	9	7.78	7.78	7.78	ATM project - testing with test case table (test cases)
105	09/10/2026	Documentation	60	3	110	3	36.67	110	3	36.67	36.67	36.67	ATM project - report and user manual (pages)

Table 2. Job Number Log for previous work. Rate = minutes per unit. Blank estimate = no estimate existed.

2.4 Notes on the estimates

I did not use PSP when I did this project, so most estimates do not exist. The only ones I had were: for coding I wrote "about 4 hours" (240 min) in my planner, and for the report the instructor asked for 3 pages and I thought it would take 1 hour (60 min). All other estimated fields are left blank on purpose. Actual times are from my Daily Time Log; LOC was counted from the final .cpp file (non-blank, non-comment lines).

3. Part 2 (ii): Student Grade Management System with PSP

The task was to develop a simple Student Grade Management System that accepts student marks, calculates the average mark and the letter grade, validates the inputs and displays the results. I kept the program small on purpose (91 LOC in Python 3) because the main goal of the assignment is following the process.

3.1 Jobs identified

Job #	Job	Process	Unit of measure	Work product
106	Requirements analysis	Requirements	requirements	Requirements list (section 3.6.1)
107	Design	Design	functions	Design (section 3.6.2)
108	Implementation	Code	LOC	grade_manager.py
109	Testing	Test	test cases	Test table + test_grade_manager.py
110	User guide and code comments	Documentation	pages	User_Guide.docx

Table 3. The five jobs of the project. Job numbers continue from Part (i).

3.2 Estimates made before each job

Before starting each job I first estimated the work-product units (how many requirements, functions, lines, test cases or pages I expected) and then multiplied by my To Date rate for that process from the Job Number Log. At that point the only data I had was from the previous project in Part (i), so the rate is the Part (i) rate.

Job #	Process	Date planned	To Date rate used	Est. units	Est. time (min)
106	Requirements	26/10/2026	5.63 min/requirement	10	10 x 5.63 = 56
107	Design	26/10/2026	13.57 min/function	7	7 x 13.57 = 95
108	Code	27/10/2026	1.54 min/LOC	110	110 x 1.54 = 169
109	Test	28/10/2026	7.78 min/test case	10	10 x 7.78 = 78
110	Documentation	28/10/2026	36.67 min/page	2	2 x 36.67 = 73

Table 4. Estimates recorded before each job started.

3.3 Daily Time Log - Student Grade Management System

Planning entries are the few minutes I spent estimating each job just before starting it. They are counted as part of that job's time.

Date	Job #	Start	Stop	Interruption (min)	Delta Time (min)	Activity	Comments	C	U
26/10/2026	106	18:30	18:45	0	15	Planning	Identified the 5 jobs + units. Estimated job 106 from my Part 1 rate		
26/10/2026	106	18:50	19:50	8	52	Requirements	Wrote FR1-FR8 and NFR1-NFR2, chose the university grading scale. Phone call	X	10
26/10/2026	107	20:30	20:36	0	6	Planning	Estimated job 107 (7 functions)		
26/10/2026	107	20:36	21:40	10	54	Design	Function list, program flow, grade scale table. Tea break		
27/10/2026	107	08:15	08:56	0	41	Design	Pseudocode for input loops. Added read_int and ask_yes_no (9 functions now)	X	9
27/10/2026	108	13:00	13:07	0	7	Planning	Estimated job 108 (~110 LOC)		
27/10/2026	108	13:07	14:35	12	76	Code	read_name, read_int, read_mark, calculate_average, letter_grade. Classmate asked about Data Mining assignment		
27/10/2026	108	19:20	20:30	4	66	Code	enter_student, show_results, main loop, table formatting. Forgot to stop timer once, corrected from laptop clock	X	91
28/10/2026	109	10:00	10:06	0	6	Planning	Estimated job 109 (10 test cases)		
28/10/2026	109	10:06	11:10	5	59	Test	Test table + unittest file. TC02 failed: used > instead of >= in grade check, fixed		
28/10/2026	109	11:20	12:05	0	45	Test	Manual input tests. TC07 crashed on 'abc' (no try/except), fixed and re-ran all 12	X	12
28/10/2026	110	15:30	15:36	0	6	Planning	Estimated job 110 (2 pages)		
28/10/2026	110	15:36	16:40	7	57	Documentation	User guide: how to run, input rules, grading scale, sample run		
28/10/2026	110	20:00	20:25	0	25	Documentation	Comments/docstrings in code, proofreading	X	2
			Total	**46**	**515**				

Table 5. Daily Time Log for Part (ii).

3.4 Job Number Log with updated To Date values

After each job I added up the Delta Time and units of that job from the Daily Time Log and updated the To Date columns. To Date includes the previous project, so for example the To Date Code rate is now based on both programs.

Job #	Date	Process	Estimated		Actual			To Date					Description (unit)
			Time	Units	Time	Units	Rate	Time	Units	Rate	Max	Min	
Part 2 (i) - Previous work: ATM Banking System (C++)													
101	05/10/2026	Requirements			45	8	5.63	45	8	5.63	5.63	5.63	ATM project - read assignment and list requirements (requirements)
102	05/10/2026	Design			95	7	13.57	95	7	13.57	13.57	13.57	ATM project - flowchart and function design (functions)
103	06/10/2026	Code	240		330	214	1.54	330	214	1.54	1.54	1.54	ATM project - coding in C++ (atm.cpp) (LOC)
104	08/10/2026	Test			70	9	7.78	70	9	7.78	7.78	7.78	ATM project - testing with test case table (test cases)
105	09/10/2026	Documentation	60	3	110	3	36.67	110	3	36.67	36.67	36.67	ATM project - report and user manual (pages)
Part 2 (ii) - Student Grade Management System (Python 3)													
106	26/10/2026	Requirements	56	10	67	10	6.70	112	18	6.22	6.70	5.63	Grade system - requirements analysis (FR/NFR list) (requirements)
107	26/10/2026	Design	95	7	101	9	11.22	196	16	12.25	13.57	11.22	Grade system - design (function list, flow, grading scale) (functions)
108	27/10/2026	Code	169	110	149	91	1.64	479	305	1.57	1.64	1.54	Grade system - implementation (grade_manager.py) (LOC)

Job #	Date	Process	Estimated		Actual			To Date					Description (unit)
			Time	Units	Time	Units	Rate	Time	Units	Rate	Max	Min	
109	28/10/2026	Test	78	10	110	12	9.17	180	21	8.57	9.17	7.78	Grade system - testing (unit tests + manual tests) (test cases)
110	28/10/2026	Documentation	73	2	88	2	44.00	198	5	39.60	44.00	36.67	Grade system - user guide and code comments (pages)

Table 6. Complete Job Number Log (jobs 101-110).

3.5 Estimated vs actual

Job #	Process	Est. time (min)	Act. time (min)	Difference (min)	Error %	Est. units	Act. units	Unit
106	Requirements	56	67	+11	+20%	10	10	requirements
107	Design	95	101	+6	+6%	7	9	functions
108	Code	169	149	-20	-12%	110	91	LOC
109	Test	78	110	+32	+41%	10	12	test cases
110	Documentation	73	88	+15	+21%	2	2	pages
	Total	**471**	**515**	**+44**	**+9%**			

Table 7. Error % = (actual - estimated) / estimated. Positive means it took longer than planned.

- Time spent on planning (estimating) entries: 40 min = 7.8% of the total Part (ii) time.
- Interruption time logged in Part (ii): 46 min (not counted in Delta Time).

3.6 Work products

3.6.1 Requirements (job 106 - 10 requirements)

ID	Requirement
FR1	The system shall ask for the student's name and shall not accept an empty name.
FR2	The system shall ask how many subjects the student has (1 to 10).
FR3	The system shall accept one mark per subject, from 0 to 100 (decimals like 72.5 allowed).
FR4	If an input is invalid (not a number, out of range or empty) the system shall show a message and ask again instead of crashing.
FR5	The system shall calculate the average mark of each student.
FR6	The system shall give a letter grade and grade point from the average using the university grading scale.
FR7	The user shall be able to enter more than one student in one run (answer y/n after each student).
FR8	The system shall display a results table (No, Name, Subjects, Average, Grade, Point) and the class average.
NFR1	The program shall be a console program in Python 3 using only the standard library.
NFR2	Averages shall be shown with 2 decimal places, and the grade shall be based on the rounded value that is shown.

Table 8. Functional (FR) and non-functional (NFR) requirements.

Grading scale used (the scale from our university academic policy):

Average mark	Letter grade	Grade point
90 - 100	A+	4.00
85 - 89.99	A	4.00
80 - 84.99	A-	3.75
75 - 79.99	B+	3.50
70 - 74.99	B	3.00
65 - 69.99	B-	2.75
60 - 64.99	C+	2.50
50 - 59.99	C	2.00
45 - 49.99	C-	1.75
40 - 44.99	D	1.00
0 - 39.99	F	0.00

Table 9. Grading scale.

3.6.2 Design (job 107 - 9 functions)

Each student is a dictionary: name, marks (list of floats), average, grade, point. All students are kept in a Python list. The grading scale is a list of (minimum, letter, point) tuples checked from the highest grade down.

Function	Input	Returns	Purpose (requirement)
main()	-	-	Program loop: enter students until the user answers n, then show results
enter_student()	-	dict	Reads one student (name, subject count, marks) and works out average and grade
read_name()	-	str	Asks for the name until it is not empty (FR1)
read_int(prompt, low, high)	prompt, limits	int	Asks for a whole number in a range, used for number of subjects (FR2, FR4)
read_mark(prompt)	prompt	float	Asks for a mark 0-100, re-asks on letters or out of range (FR3, FR4)
calculate_average(marks)	list of marks	float	Average rounded to 2 decimals (FR5, NFR2)
letter_grade(average)	average	(letter, point)	Looks up the grading scale from the top (FR6)
ask_yes_no(prompt)	prompt	bool	y/n question for adding another student (FR7)
show_results(students)	list of students	-	Prints the results table and class average (FR8)

Table 10. Functions designed.

Program flow:

1. Start: print the title.
2. Repeat: enter_student() -> read_name(), read_int() for the number of subjects, read_mark() for each subject.
3. calculate_average(marks) and letter_grade(average); store the student as a dictionary in a list.
4. ask_yes_no("Add another student?"): y = repeat, n = continue.
5. show_results(list): table + class average. End.

3.6.3 Implementation (job 108 - 91 LOC)

The program is in source/grade_manager.py (91 LOC, counted as non-blank lines that are not comments or docstrings, counted on the final file, so it includes the try/except lines added for defect D2 during testing). It only uses the Python standard library. The full listing is in Part D of this document and in the GitHub repository. The unit test file test_grade_manager.py was written during testing, so its lines are not counted as Code LOC.

3.6.4 Testing (job 109 - 12 test cases)

TC01-TC06 are automated in test_grade_manager.py (python -m unittest). TC07-TC12 were run by hand by typing the inputs into the program.

ID	Test / input	Expected result	Actual result	Result
TC01	calculate_average([80, 90, 70])	80.00	80.00	Pass
TC02	letter_grade(90) - boundary	A+, 4.00	A+, 4.00	Pass (failed on first run - defect D1, fixed)
TC03	letter_grade(89.99)	A, 4.00	A, 4.00	Pass
TC04	letter_grade(40) - lowest pass	D, 1.00	D, 1.00	Pass
TC05	letter_grade(39.5)	F, 0.00	F, 0.00	Pass
TC06	letter_grade(100) and letter_grade(0)	A+ and F	A+ and F	Pass
TC07	Mark typed as "abc", then 70	Message, asks again, accepts 70	"Mark must be a number..." then accepted 70	Pass (crashed on first run - defect D2, fixed)
TC08	Mark 105, then 70	Message, asks again	"Mark must be between 0 and 100."	Pass
TC09	Mark -5, then 70	Message, asks again	"Mark must be between 0 and 100."	Pass
TC10	Empty name and spaces-only name	Message, asks again	"Name cannot be empty. Try again." (twice)	Pass
TC11	Subjects 0, 11, "two", then 2	Messages, then accepts 2	Range message twice, "Please enter a whole number."	Pass
TC12	Full run: Abebe 86.5/92/78, Sara 55/62/48, answer "maybe" then n	85.50 A 4.00; 55.00 C 2.00; class 70.25	85.50 A 4.00; 55.00 C 2.00; class 70.25	Pass

Table 11. Test cases and results (final run).

Defects found during testing:

No.	Found by	Defect	Fix
D1	TC02	letter_grade used > instead of >=, so an average of exactly 90 got A instead of A+.	Changed to >=. Re-ran TC01-TC06.
D2	TC07	read_mark did float(input()) without try/except, so typing letters crashed the program with ValueError.	Added try/except with a message and loop. Re-ran all 12 tests.

Table 12. Defects found and fixed.

Output of the automated tests (python -m unittest -v):

```
test_tc01_average (test_grade_manager.TestCalculations.test_tc01_average) ... ok
test_tc02_boundary_a_plus (test_grade_manager.TestCalculations.test_tc02_boundary_a_plus) ... ok
test_tc03_just_below_boundary (test_grade_manager.TestCalculations.test_tc03_just_below_boundary) ... ok
test_tc04_lowest_pass (test_grade_manager.TestCalculations.test_tc04_lowest_pass) ... ok
test_tc05_fail (test_grade_manager.TestCalculations.test_tc05_fail) ... ok
test_tc06_limits (test_grade_manager.TestCalculations.test_tc06_limits) ... ok

----------------------------------------------------------------------
Ran 6 tests in 0.000s

OK
```

3.6.5 Documentation (job 110 - 2 pages)

The user guide is a separate file, User_Guide.docx (2 pages), reproduced in Part B. It explains how to run the program, the input rules, the grading scale and shows a sample run. I also added comments to the code during this job.

4. Reflection: did PSP make my workflow better or worse?

Overall result

Before this course I never planned my programming time. In Part (i) the only guesses I had were "about 4 hours" for coding and 1 hour for the report, and they were wrong by +38% and +83%. For the grade system I estimated every job from my Part (i) rates. In total I estimated 471 minutes and it took 515 minutes, so I was +9% over. That is much closer than my old guesses, even though some single jobs were still far off.

What PSP made better

- My estimates had a reason behind them. Instead of guessing, I multiplied the planned units by my To Date rate. Design (+6%) and coding (-12%) came out close.
- Logging interruptions showed me where the time really goes. I wrote down 46 minutes of interruptions in Part (ii) (phone calls, tea, a classmate asking about another course). Before, I would have counted those minutes as work.
- Because I had to estimate the number of test cases before testing, I wrote the test table first. That is how I found the two defects (D1 and D2) in a planned way and not by luck.
- Splitting the project into jobs made it easier to continue after a break, because I always knew which job I was on and what was left.

What PSP made worse

- It costs time. The planning entries alone took 40 minutes, which is 7.8% of the 515 minutes, and writing every start and stop time adds more. For a program of 91 lines this felt like a lot.
- Stopping the timer is easy to forget. On 27 October I forgot once and had to correct the stop time from my laptop clock, so that entry is not exact.
- Units for requirements, design and documentation are rough. One requirement or one page is not always the same amount of work, so those rates are less reliable than LOC.
- My code rate came from a C++ project. Python needs fewer lines for the same job, so I planned 110 LOC and wrote only 91, and my minutes per LOC went up (1.54 before, 1.64 now).
- I still underestimated testing (+41%) and documentation (+21%), the same kind of jobs I underestimated in Part (i). The difference is that now I have data that shows it.

My conclusion

For me PSP made the work a little slower (about 8% extra time for planning) but much more predictable, so overall I think it helped. I will keep using the Daily Time Log and the Job Number Log. To reduce the overhead I will plan faster (the first plan took 15 minutes, the later ones 6-7 minutes), and for testing and documentation I will use the To Date Max rate instead of the average rate, because those are the jobs I keep underestimating.

Part B. User Guide

Student Grade Management System

1. What the program does

The program lets a teacher type the marks of one or more students. For each student it calculates the average mark, the letter grade and the grade point, and at the end it prints a results table with the class average. Wrong inputs are not accepted; the program explains the problem and asks again.

2. What you need

- Python 3.8 or newer (no extra packages).
- The file grade_manager.py (and test_grade_manager.py to run the tests).

3. How to run it

Open a terminal / command prompt in the source folder and type:

```
python grade_manager.py
```

4. How to use it

1. Type the student's name and press Enter.
2. Type how many subjects the student has (1 to 10).
3. Type the mark for each subject (0 to 100, decimals like 72.5 are fine).
4. Answer y to add another student or n to finish.
5. The results table and the class average are printed and the program ends.

5. Input rules

Input	Allowed	If it is wrong
Student name	Any text, not empty	"Name cannot be empty. Try again."
Number of subjects	Whole number 1-10	"Number must be between 1 and 10." or "Please enter a whole number."
Mark	Number 0-100	"Mark must be a number..." or "Mark must be between 0 and 100."
Add another student?	y / yes / n / no	"Please answer y or n."

6. Grading scale

The average is rounded to 2 decimal places first, and the grade is taken from the rounded average.

Average mark	Grade	Point
90 - 100	A+	4.00
85 - 89.99	A	4.00
80 - 84.99	A-	3.75
75 - 79.99	B+	3.50
70 - 74.99	B	3.00
65 - 69.99	B-	2.75
60 - 64.99	C+	2.50
50 - 59.99	C	2.00
45 - 49.99	C-	1.75
40 - 44.99	D	1.00
0 - 39.99	F	0.00

7. Sample run

Typed inputs are shown after each prompt:

```
=== Student Grade Management System ===

Student #1
Student name: Abebe Kebede
Number of subjects (1-10): 3
  Mark for subject 1: 86.5
  Mark for subject 2: 92
  Mark for subject 3: 78
Add another student? (y/n): y

Student #2
Student name: Sara Tesfaye
Number of subjects (1-10): 3
  Mark for subject 1: 55
  Mark for subject 2: 62
  Mark for subject 3: 48
Add another student? (y/n): n

RESULTS
------------------------------------------------------------
No  Name                 Subjects  Average  Grade  Point
------------------------------------------------------------
1   Abebe Kebede                3    85.50      A   4.00
2   Sara Tesfaye                3    55.00      C   2.00
------------------------------------------------------------
Class average: 70.25
```

8. Running the tests

The calculation tests (TC01-TC06) can be run with:

```
python -m unittest test_grade_manager.py
```

All 6 tests should print "ok" and the last line should be OK.

Part C. PSP Logs (PSP_Logs.xlsx)

The three sheets of the Excel workbook. Delta Time, actual time and units, rates and the To Date columns are formulas in the workbook; the values below are the ones those formulas give.

C.1 Sheet "Job Number Log"

Rate = minutes per unit. Actual time and units are added up from the Daily Time Log. To Date = all jobs so far of the same process. Blank estimate = no estimate existed at the time.

Job #	Date	Process	Estimated		Actual			To Date					Description
			Time (min)	Units	Time (min)	Units	Rate (min/unit)	Time (min)	Units	Rate (min/unit)	Max	Min	
Part 2 (i) - Previous work: ATM Banking System (C++)													
101	05/10/2026	Requirements			45	8	5.63	45	8	5.63	5.63	5.63	ATM project - read assignment and list requirements (requirements)
102	05/10/2026	Design			95	7	13.57	95	7	13.57	13.57	13.57	ATM project - flowchart and function design (functions)
103	06/10/2026	Code	240		330	214	1.54	330	214	1.54	1.54	1.54	ATM project - coding in C++ (atm.cpp) (LOC)
104	08/10/2026	Test			70	9	7.78	70	9	7.78	7.78	7.78	ATM project - testing with test case table (test cases)
105	09/10/2026	Documentation	60	3	110	3	36.67	110	3	36.67	36.67	36.67	ATM project - report and user manual (pages)
Part 2 (ii) - Student Grade Management System (Python 3)													
106	26/10/2026	Requirements	56	10	67	10	6.70	112	18	6.22	6.70	5.63	Grade system - requirements analysis (FR/NFR list) (requirements)
107	26/10/2026	Design	95	7	101	9	11.22	196	16	12.25	13.57	11.22	Grade system - design (function list, flow, grading scale) (functions)
108	27/10/2026	Code	169	110	149	91	1.64	479	305	1.57	1.64	1.54	Grade system - implementation (grade_manager.py) (LOC)
109	28/10/2026	Test	78	10	110	12	9.17	180	21	8.57	9.17	7.78	Grade system - testing (unit tests + manual tests) (test cases)
110	28/10/2026	Documentation	73	2	88	2	44.00	198	5	39.60	44.00	36.67	Grade system - user guide and code comments (pages)

Units of measure: Requirements = requirements; Design = functions; Code = LOC; Test = test cases; Documentation = pages. LOC = non-blank lines that are not comments.

C.2 Sheet "Daily Time Log"

Times in minutes. Delta Time = Stop - Start - Interruption. C = job completed, U = units completed.

Date	Job #	Start	Stop	Interruption (min)	Delta Time (min)	Activity	Comments	C	U
Part 2 (i) - Previous work: ATM Banking System (C++)									
05/10/2026	101	19:10	19:58	3	45	Requirements	Read the assignment sheet, listed the ATM features (PIN, balance, deposit, withdraw...)	X	8
05/10/2026	102	20:05	20:50	0	45	Design	Flowchart for main menu and PIN check		
06/10/2026	102	18:30	19:30	10	50	Design	Function list + pseudocode for deposit/withdraw. Phone call from home	X	7
06/10/2026	103	19:40	21:15	5	90	Code	Account struct, menu loop, PIN check		
07/10/2026	103	14:00	15:50	15	95	Code	deposit / withdraw / balance functions. Friend came to borrow notes		
07/10/2026	103	20:10	21:30	0	80	Code	Transaction history (array of last 5), fixed compile errors		
08/10/2026	103	09:00	10:20	15	65	Code	Input checks, change PIN, cleaned up code. Break for tea	X	214
08/10/2026	104	10:30	11:45	5	70	Test	Ran 9 test cases from my test table. Fixed: withdraw accepted a negative amount	X	9
09/10/2026	105	16:00	17:05	0	65	Documentation	Report: introduction, features, screenshots		
09/10/2026	105	20:00	20:50	5	45	Documentation	User manual part + formatting	X	3
			Total	**58**	**650**				
Part 2 (ii) - Student Grade Management System (Python 3)									
26/10/2026	106	18:30	18:45	0	15	Planning	Identified the 5 jobs + units. Estimated job 106 from my Part 1 rate		
26/10/2026	106	18:50	19:50	8	52	Requirements	Wrote FR1-FR8 and NFR1-NFR2, chose the university grading scale. Phone call	X	10
26/10/2026	107	20:30	20:36	0	6	Planning	Estimated job 107 (7 functions)		
26/10/2026	107	20:36	21:40	10	54	Design	Function list, program flow, grade scale table. Tea break		
27/10/2026	107	08:15	08:56	0	41	Design	Pseudocode for input loops. Added read_int and ask_yes_no (9 functions now)	X	9
27/10/2026	108	13:00	13:07	0	7	Planning	Estimated job 108 (~110 LOC)		
27/10/2026	108	13:07	14:35	12	76	Code	read_name, read_int, read_mark, calculate_average, letter_grade. Classmate asked about Data Mining assignment		
27/10/2026	108	19:20	20:30	4	66	Code	enter_student, show_results, main loop, table formatting. Forgot to stop timer once, corrected from laptop clock	X	91
28/10/2026	109	10:00	10:06	0	6	Planning	Estimated job 109 (10 test cases)		
28/10/2026	109	10:06	11:10	5	59	Test	Test table + unittest file. TC02 failed: used > instead of >= in grade check, fixed		
28/10/2026	109	11:20	12:05	0	45	Test	Manual input tests. TC07 crashed on 'abc' (no try/except), fixed and re-ran all 12	X	12
28/10/2026	110	15:30	15:36	0	6	Planning	Estimated job 110 (2 pages)		
28/10/2026	110	15:36	16:40	7	57	Documentation	User guide: how to run, input rules, grading scale, sample run		
28/10/2026	110	20:00	20:25	0	25	Documentation	Comments/docstrings in code, proofreading	X	2
			Total	**46**	**515**				

C.3 Sheet "Part 2 Summary"

Error % = (Actual - Estimated) / Estimated. Positive = took longer than planned.

Job #	Process	Est. Time (min)	Act. Time (min)	Difference (min)	Time Error %	Est. Units	Act. Units	Unit
106	Requirements	56	67	11	+20%	10	10	requirements
107	Design	95	101	6	+6%	7	9	functions
108	Code	169	149	-20	-12%	110	91	LOC
109	Test	78	110	32	+41%	10	12	test cases
110	Documentation	73	88	15	+21%	2	2	pages
	Total	**471**	**515**	**44**	**+9%**			

Time spent on PSP planning (Planning entries)	40
Planning as % of total Part 2 time	7.8%
Interruption time logged in Part 2 (min)	46

Part D. Source Code

Student Grade Management System (Python 3)

Repository: https://github.com/Zelalem-Fisseha/Zelalem_Fisseha_Bits_UGR_0177
Branch: main
Commit: `bed64859d7193fab7da92177f49d1a579c6d2988`
The listings below are copied from that commit.

Repository contents

`source/grade_manager.py`	The program (job 108)
`source/test_grade_manager.py`	Unit tests TC01-TC06 (job 109)
`PSP_Report.docx`	The PSP report (Part A)
`User_Guide.docx`	The user guide (Part B)
`PSP_Logs.xlsx`	Daily Time Log, Job Number Log and summary (Part C)

How to get and run the code

Needs Python 3.8 or newer, no extra packages.

```
git clone https://github.com/Zelalem-Fisseha/Zelalem_Fisseha_Bits_UGR_0177.git
cd Zelalem_Fisseha_Bits_UGR_0177/source
python grade_manager.py                      # run the program
python -m unittest -v test_grade_manager.py  # run TC01-TC06
```

D.1 source/grade_manager.py

```
# Student Grade Management System
# PSP Assignment - Part 2 (ii)
#
# Reads marks for one or more students, calculates the average mark,
# gives a letter grade (and grade point) and prints a results table.
# Run with:  python grade_manager.py

MAX_SUBJECTS = 10

# Grading scale used in our university: (minimum average, letter, grade point)
# Checked from the top, so the first row the average reaches is the grade.
GRADE_SCALE = [
    (90, "A+", 4.00),
    (85, "A", 4.00),
    (80, "A-", 3.75),
    (75, "B+", 3.50),
    (70, "B", 3.00),
    (65, "B-", 2.75),
    (60, "C+", 2.50),
    (50, "C", 2.00),
    (45, "C-", 1.75),
    (40, "D", 1.00),
    (0, "F", 0.00),
]


def read_name():
    """Ask for a student name until something is typed."""
    while True:
        name = input("Student name: ").strip()
        if name != "":
            return name
        print("  Name cannot be empty. Try again.")


def read_int(prompt, low, high):
    """Ask for a whole number between low and high (inclusive)."""
    while True:
        text = input(prompt).strip()
        try:
            value = int(text)
        except ValueError:
            print("  Please enter a whole number.")
            continue
        if low <= value <= high:
            return value
        print(f"  Number must be between {low} and {high}.")


def read_mark(prompt):
    """Ask for a mark out of 100. Decimals like 72.5 are allowed."""
    while True:
        text = input(prompt).strip()
        try:
            mark = float(text)
        except ValueError:
            print("  Mark must be a number, e.g. 75 or 82.5")
            continue
        if 0 <= mark <= 100:
            return mark
        print("  Mark must be between 0 and 100.")


def calculate_average(marks):
    # rounded to 2 decimals so the grade matches the number that is printed
    return round(sum(marks) / len(marks), 2)


def letter_grade(average):
```

```
    """Return (letter, grade point) for an average mark."""
    for minimum, letter, point in GRADE_SCALE:
        if average >= minimum:
            return letter, point
    return "F", 0.00


def ask_yes_no(prompt):
    while True:
        answer = input(prompt).strip().lower()
        if answer in ("y", "yes"):
            return True
        if answer in ("n", "no"):
            return False
        print("  Please answer y or n.")


def enter_student():
    """Read one student's name and marks, and work out the result."""
    name = read_name()
    count = read_int(f"Number of subjects (1-{MAX_SUBJECTS}): ", 1, MAX_SUBJECTS)
    marks = []
    for i in range(1, count + 1):
        marks.append(read_mark(f"  Mark for subject {i}: "))
    average = calculate_average(marks)
    letter, point = letter_grade(average)
    return {"name": name, "marks": marks, "average": average,
            "grade": letter, "point": point}


def show_results(students):
    line = "-" * 56
    print()
    print("RESULTS")
    print(line)
    print(f"{'No':<4}{'Name':<20}{'Subjects':>9}{'Average':>9}{'Grade':>7}{'Point':>7}")
    print(line)
    for i, s in enumerate(students, start=1):
        print(f"{i:<4}{s['name']:<20}{len(s['marks']):>9}"
              f"{s['average']:>9.2f}{s['grade']:>7}{s['point']:>7.2f}")
    print(line)
    class_average = calculate_average([s["average"] for s in students])
    print(f"Class average: {class_average:.2f}")


def main():
    print("=== Student Grade Management System ===")
    students = []
    while True:
        print(f"\nStudent #{len(students) + 1}")
        students.append(enter_student())
        if not ask_yes_no("Add another student? (y/n): "):
            break
    show_results(students)


if __name__ == "__main__":
    main()
```

D.2 source/test_grade_manager.py

```
# Unit tests for the calculation part of grade_manager.py
# (test cases TC01 - TC06 in the test table). Run with:
#   python -m unittest test_grade_manager.py

import unittest

from grade_manager import calculate_average, letter_grade


class TestCalculations(unittest.TestCase):

    def test_tc01_average(self):
        self.assertEqual(calculate_average([80, 90, 70]), 80.00)

    def test_tc02_boundary_a_plus(self):
        self.assertEqual(letter_grade(90), ("A+", 4.00))

    def test_tc03_just_below_boundary(self):
        self.assertEqual(letter_grade(89.99), ("A", 4.00))

    def test_tc04_lowest_pass(self):
        self.assertEqual(letter_grade(40), ("D", 1.00))

    def test_tc05_fail(self):
        self.assertEqual(letter_grade(39.5), ("F", 0.00))

    def test_tc06_limits(self):
        self.assertEqual(letter_grade(100), ("A+", 4.00))
        self.assertEqual(letter_grade(0), ("F", 0.00))


if __name__ == "__main__":
    unittest.main()
```